Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.10.3 sur 153 | difficulté 0 / 10

# 0.10.3. La fonction textS() — un texte qui passe à la ligne

**#include <textS> : comme texte(), mais arrivé au bord droit, la suite repart en colonne 0 de la ligne d'en dessous.**

## 1. Ce qu'il faut comprendre

**Ce qu'elle fait :** `textS(colonne, ligne, "TEXTE")` écrit comme `texte()`, mais **ne s'arrête pas au bord** : après la colonne 19, la lettre suivante va en colonne 0 de la ligne d'en dessous. Après la ligne 17, elle remonte en haut.

**La ligne à écrire :** `#include <textS>`.

**Ses arguments :** les mêmes que `texte()` : colonne, ligne, texte entre guillemets.

**Ce qu'elle coûte :** environ **250 octets** la première fois : elle calcule la case de chaque lettre pendant le jeu.

**Essaie :** `textS(0, 0, "...")` avec un texte de 45 lettres : il remplit deux lignes et demie.

> **On retrouve textS dans 3 leçons**
>
> 0.11, 0.11.1, 0.11.2

### 📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

### 🔍 Le programme, ligne par ligne — chaque ligne dite en français

| | |
|---|---|
| `// ---- #include <textS> : un texte qui passe à la ligne ----` | Un commentaire : une phrase pour celui qui lit (« ---- #include <textS> : un texte qui passe à la ligne ---- »). La console l'ignore. |
| `// BONJOUR commence en colonne 15 : B O N J O tiennent jusqu'à la colonne 19,` | Un commentaire : une phrase pour celui qui lit (« BONJOUR commence en colonne 15 : B O N J O tiennent jusqu'à la colonne 19, »). La console l'ignore. |
| `// puis U et R repartent en colonne 0, une ligne plus bas.` | Un commentaire : une phrase pour celui qui lit (« puis U et R repartent en colonne 0, une ligne plus bas. »). La console l'ignore. |
| `#include <textS>  // textS() : le texte qui va à la ligne tout seul` | Demande à la console « textS » (ce que c'est : écrit un texte qui passe à la ligne tout seul). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « textS() : le texte qui va à la ligne tout seul ».) |
| `int main() {` | Le début du programme : la console commence ici, par la première ligne qui suit. |
| `textS(15, 8, "BONJOUR");  // BONJO en (15..19, 8), puis UR en (0..1, 9)` | Appel de la fonction textS de la console, avec les arguments 15, 8, "BONJOUR". Ce qu'elle fait : comme texte(), mais passe à la ligne |

| | |
|---|---|
| | tout seul : après la colonne 19, la suite reprend en colonne 0 de la ligne d'en dessous ; après la ligne 17, en haut. Un 4e argument est une TAILLE, sans limite, comme pour texte() : textS(0, 0, "BONJOUR", 2). (Le commentaire le dit : « BONJO en (15..19, 8), puis UR en (0..1, 9) ».) |
| `while (true) {` | La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). |
| `image();` | Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. |
| `}` | L'accolade fermante : la fin du bloc ouvert plus haut par « { ». |
| `}` | L'accolade fermante : la fin du bloc ouvert plus haut par « { ». |

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **336 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           29 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     67 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           96 o  ← CopierTuiles  — 6 lettres sur 43
  "BONJO"           5 o  ← texte()  — un texte gravé
  "UR"              2 o  ← texte()  — un texte gravé
Lettres gravées : BJNORU
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 2    22 o sur place, dans main()  + EcrireTexte (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  textS() × 1     0 o sur place, dans main()
```

#### 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <textS> : un texte qui passe à la ligne ----
// BONJOUR commence en colonne 15 : B O N J O tiennent jusqu'à la colonne 19,
// puis U et R repartent en colonne 0, une ligne plus bas.

#include <textS>   // textS() : le texte qui va à la ligne tout seul
```

#### 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  textS(15, 8, "BONJOUR");   // BONJO en (15..19, 8), puis UR en (0..1, 9)

  while (true) {
    image();
  }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1




un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

> BONJO tout à droite de la ligne 8, et UR au début de la ligne 9.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ BONJO tient jusqu'au bord droit
- ✓ UR repart en colonne 0, une ligne plus bas (« UR »)

---

Ensuite : **0.11. Tout l'alphabet, avec textS** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++